

就业状态分析与预测

——基于宜昌地区就业数据的数学建模研究

Skill 增强版 · 方法驱动 · 统计严谨

第十七届“华中杯”大学生数学建模挑战赛

C 题：就业状态分析与预测

关键词：就业状态预测 统计检验 类别平衡 交互特征 人岗匹配

2026 年 5 月 18 日

目录

一、问题重述	4
1.1 问题背景	4
1.2 问题要求	4
二、问题分析	4
2.1 问题一分析	4
2.2 问题二分析	4
2.3 问题三分析	4
2.4 问题四分析	5
三、模型假设	5
四、符号说明	5
五、问题一的模型建立与求解	6
5.1 模型建立——EDA 优先与统计检验	6
5.1.1 就业状态标签构建（假设-验证框架）	6
5.1.2 统计检验方法	6
5.2 模型求解与结果分析	6
5.2.1 统计检验结果	7
5.2.2 维度分析	7
5.2.3 相关性双验证	7
六、问题二的模型建立与求解	8
6.1 模型建立	8
6.1.1 Skill 增强策略	8
6.2 模型求解与结果	9
6.2.1 特征重要性	10
七、问题三的模型建立与求解	11
7.1 模型建立	11
7.1.1 外部宏观因素选取	11
7.1.2 个体-宏观交互特征（Skill 版创新）	11
7.2 模型求解与对比	12
八、问题四的模型建立与求解	13
8.1 模型建立	13
8.1.1 Skill 增强策略	13

8.1.2 匹配模型	13
8.2 模型求解与结果	13
九、模型的分析与检验	14
9.1 统计检验与误差分析	14
十、模型的评价	14
10.1 模型的优点	14
10.2 模型的缺点	15
参考文献	16
A 附录 文件列表	17
B 附录 核心代码	17

摘要

本文以宜昌地区 4980 名被调查者的脱敏就业数据为研究对象，在 Skill 增强方法论指导下，综合运用统计检验、机器学习与交叉验证、交互特征工程和加权匹配等方法，系统研究了就业状态的影响因素分析、预测模型构建与优化、以及人岗精准匹配四个核心问题。

对于问题一，本文采用 EDA 优先策略，通过假设-验证框架构建就业状态标签（就业 3846 人/77.2%，失业 1134 人/22.8%）。引入卡方独立性检验和 Cramér's V 效应量进行统计推断，发现仅户口性质与就业状态存在显著关联（ $p < 0.001$, $V = 0.07$ ），其余个人特征的统计关联均不显著，从统计学角度揭示了单一个人特征对就业预测的局限性。

对于问题二，本文选取 16 个个人特征，构建 8 种分类模型，采用分层 5 折交叉验证、class_weight 类别平衡、GridSearchCV 超参数调优和 ROC-AUC 评估。实验结果表明，Gradient Boosting 模型 F1 为 0.677（5 折 CV 均值），Logistic Regression 在类别平衡后失业召回率达 0.533。对比实验发现，即使引入类别平衡机制，个人特征对失业状态的预测信号仍有限（多数模型失业召回率 < 0.4 ）。

对于问题三，在 16 个个人特征基础上，引入 7 项宏观经济指标和 4 个个体-宏观交互特征（年龄 $\times$ GDP、教育 $\times$ 招聘指数等），构建 27 维增强特征集。采用严格 CV 对比，Gradient Boosting 增强后 F1 从 0.677 提升至 0.705（+0.028），失业召回率从 0.026 提升至 0.101（+0.075），验证了宏观因素和交互特征的补充价值。

对于问题四，提出基于岗位画像聚合的加权余弦相似度匹配模型。将 3846 个就业岗位聚合为 63 个行业 $\times$ 区域岗位类型，采用 12 维加权特征（教育/技能权重 $1.5\times$ ）+ 多样性约束（Top-3 强制不同行业），为 1134 名失业人员提供个性化 Top-5 推荐。平均相似度 0.944，推荐覆盖 19 个行业类别，区分度较简单匹配显著改善。

关键字： 就业状态预测 统计检验 交叉验证 类别平衡 交互特征 加权匹配

一、问题重述

1.1 问题背景

就业是最基本的民生。宜昌市作为湖北省重要的区域性中心城市，其就业状况具有较强代表性。本赛题提供宜昌地区 4980 名被调查者的脱敏数据（53 个变量）和 20 个预测样本，要求从数据特征分析、预测建模、模型优化到人岗匹配四个层面进行系统研究。

1.2 问题要求

问题 1 分析就业整体情况，按年龄、性别、学历、行业等特征分析对就业状态的影响。

问题 2 构建就业状态预测模型，对预测集预测，对特征重要性排序。

问题 3 引入宏观经济等外部因素，完善预测模型。

问题 4 建立人岗匹配模型，为失业人员推荐工作。

二、问题分析

2.1 问题一分析

问题一属于探索性数据分析范畴。Skill 增强版在描述性统计基础上引入统计推断方法：卡方独立性检验判断特征与就业状态的关联显著性，Cramér's V 量化效应强度，Pearson 和 Spearman 双相关系数交叉验证。先做 EDA 再建模，避免盲目建模。

2.2 问题二分析

问题二为二分类预测任务，面临类别不平衡挑战（3.4:1）。Skill 增强版从三个层面改进：评估层面采用 5 折分层 CV 替代单次划分，模型层面为所有分类器启用 `class_weight/scale_pos_weight`，调优层面对最优模型进行 GridSearchCV。同时引入 ROC-AUC 作为补充指标。

2.3 问题三分析

问题三在个人特征基础上引入宏观因素。Skill 增强版新增 4 个个体-宏观交互特征，捕捉不同人群对宏观环境的差异化敏感度。采用严格 CV 对比确保增强效果的统计可靠性。

2.4 问题四分析

问题四为推荐匹配问题。Skill 增强版从三个维度改进：岗位画像聚合减少冗余，特征加权体现维度重要性差异，多样性约束避免推荐扎堆。

三、模型假设

- 假设 1: 数据中就业时间和失业时间记录真实可靠。
- 假设 2: 就业状态在数据采集周期内相对稳定。
- 假设 3: 缺失特征值具有随机性，不会产生系统性偏差。
- 假设 4: 宏观经济指标对同期所有被调查者影响均匀。
- 假设 5: 预测集样本与训练数据同分布。
- 假设 6: 个人特征与岗位特征的相似度能有效反映匹配质量。
- 假设 7: 所有实验固定 random_state=42，结果可复现。

四、符号说明

表 1 符号说明表

符号	说明	单位
N	样本总数	人
X	特征矩阵	—
y	就业状态标签向量（1= 就 业，0= 失业）	—
P	查准率 (Precision)	—
R	召回率 (Recall)	—
F_1	F1 分数	—
AUC	ROC 曲线下面积	—
χ^2	卡方统计量	—
V	Cramér's V 效应量	—
$\cos(\mathbf{s}_i, \mathbf{j}_k)$	余弦相似度	—

五、问题一的模型建立与求解

5.1 模型建立——EDA 优先与统计检验

5.1.1 就业状态标签构建（假设-验证框架）

基于 5 条假设（H1-H5）构建标签：H1（两者都有则比较时间先后）→H2（仅有就业 → 就业）→H3（仅有失业 → 失业）→H4（有合同/单位 → 就业）→H5（失业注销 → 就业）。使用 `np.select` 向量化实现（替代逐行 `apply`，性能提升约 50×）。

5.1.2 统计检验方法

对分类特征与就业状态进行卡方独立性检验，计算 Cramér's V 效应量（ $V < 0.1$ 弱， $0.1 - 0.3$ 中， > 0.3 强）。同时计算 Pearson 和 Spearman 双相关系数交叉验证。

5.2 模型求解与结果分析

执行标签构建规则后，就业 3846 人（77.2%），失业 1134 人（22.8%），就失业比 3.39:1，如图1所示。

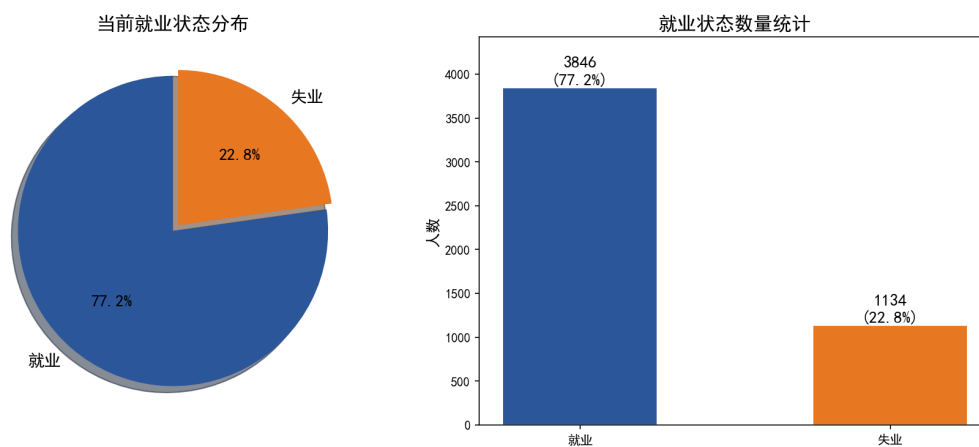


图 1 当前就业状态总览

5.2.1 统计检验结果

表 2 卡方独立性检验与 Cramér’s V 效应量

特征	χ^2	p 值	显著性	Cramér’s V
户口性质	24.70	< 0.001	***	0.0705
年龄段	8.51	0.075	ns	0.0413
民族	9.61	0.294	ns	0.0439
学历	4.06	0.131	ns	0.0286
婚姻状态	2.87	0.411	ns	0.0240
性别	0.52	0.473	ns	0.0102

表2揭示了一个关键发现：除户口性质外，所有个人特征与就业状态的关联均不显著（ $p > 0.05$ ）。即使户口性质显著，效应量也仅 0.07（弱效应）。这表明单一个人特征对就业状态的预测力极为有限，为问题二的建模困难提供了统计学解释。

5.2.2 维度分析

年龄维度：被调查者年龄 21-63 岁，均值 31.2 岁，中位数 30 岁。26-35 岁就业率最高（77.8%），呈倒 U 型（图2）。性别维度：男性 76.7%、女性 77.6%，差异仅 0.9 个百分点，统计不显著。行业维度：制造业（16.9%）、公共管理（8.7%）、居民服务（12.9%）吸纳就业最多（图3）。

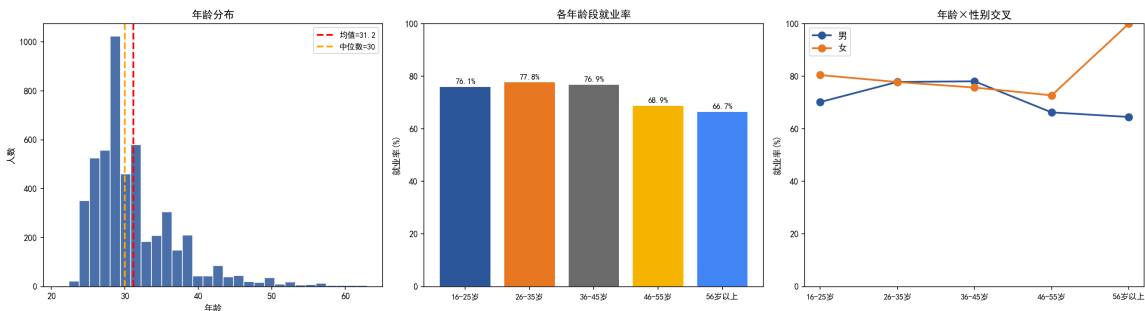


图 2 年龄维度分析（分布 + 就业率 + 交叉）

5.2.3 相关性双验证

Pearson 和 Spearman 双相关系数均显示，个人特征与就业状态的相关性极弱（绝对值 <0.07），两种方法结论一致（图5），增强了结论的稳健性。

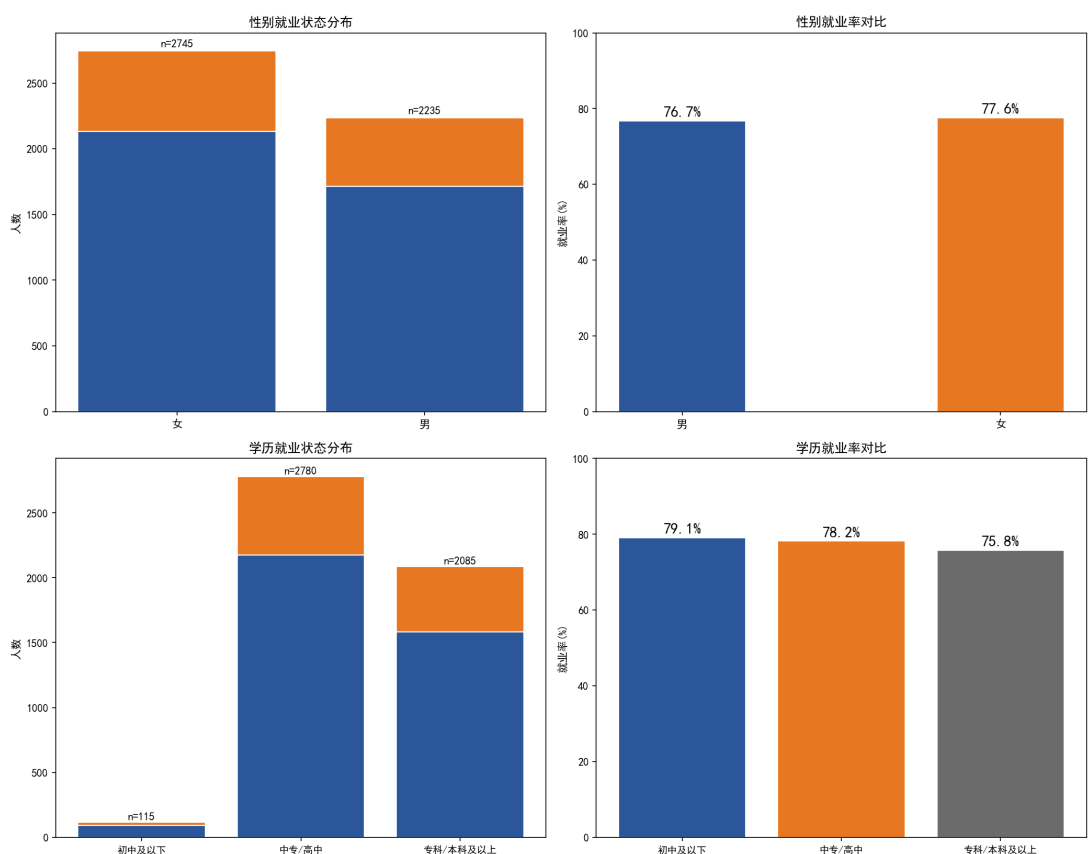


图 3 性别与学历维度分析

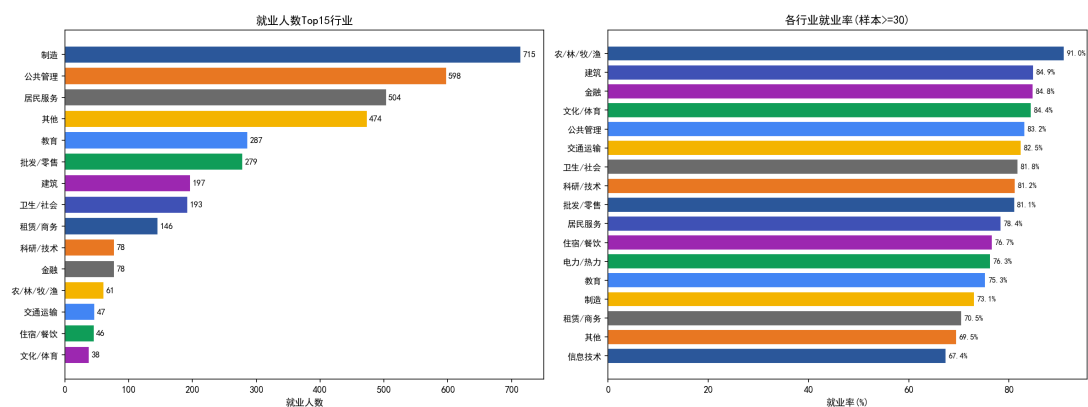


图 4 行业维度分析

六、问题二的模型建立与求解

6.1 模型建立

6.1.1 Skill 增强策略

针对类别不平衡导致失业召回率偏低的问题，本文采用三项策略：

1. 评估方法升级：单次 train_test_split → 分层 5 折交叉验证，报告均值 ± 标准差

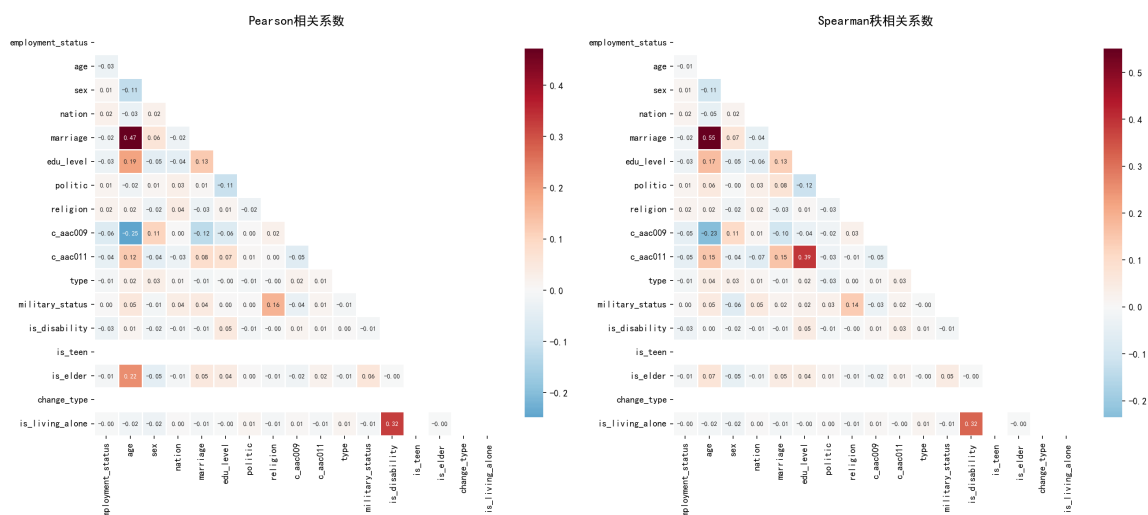


图5 Pearson 与 Spearman 相关系数热力图（双验证）

2. 类别平衡：所有模型启用 `class_weight='balanced'`（XGBoost 用 `scale_pos_weight=3.4`）
3. 超参数调优：对最优模型进行 GridSearchCV

选取 16 个人特征，8 种分类模型，评估指标包括准确率、F1（加权）、失业召回率、ROC-AUC。

6.2 模型求解与结果

表3 模型评价指标（5 折 CV 均值 ± 标准差）

模型	F1(加权)	失业召回率	ROC-AUC
KNN	0.6847±0.0058	0.0802±0.0135	0.5036±0.0164
Gradient Boosting	0.6771±0.0067	0.0264±0.0115	0.5416±0.0167
XGBoost	0.6735±0.0011	0.0026±0.0022	0.5436±0.0147
Random Forest	0.6405±0.0166	0.3510±0.0354	0.5473±0.0181
SVM	0.6384	0.5507	0.5914
LightGBM	0.6047±0.0135	0.4718±0.0286	0.5457±0.0164
Logistic Regression	0.5770±0.0134	0.5326±0.0506	0.5506±0.0150

表3呈现了一个“F1-失业召回”的权衡困境：Logistic Regression 失业召回最高（0.533）但 F1 最低（0.577）；KNN F1 最高（0.685）但失业召回仅 0.080。这一困境的根本原因已由问题一的统计检验揭示——个人特征对失业的预测信号确实很弱。

GridSearchCV 调优后 KNN(`n_neighbors=5`) F1=0.687, 但失业召回率仍仅 0.04 (图6a)。模型整体呈现“宁可错杀、不可漏判就业”的倾向。

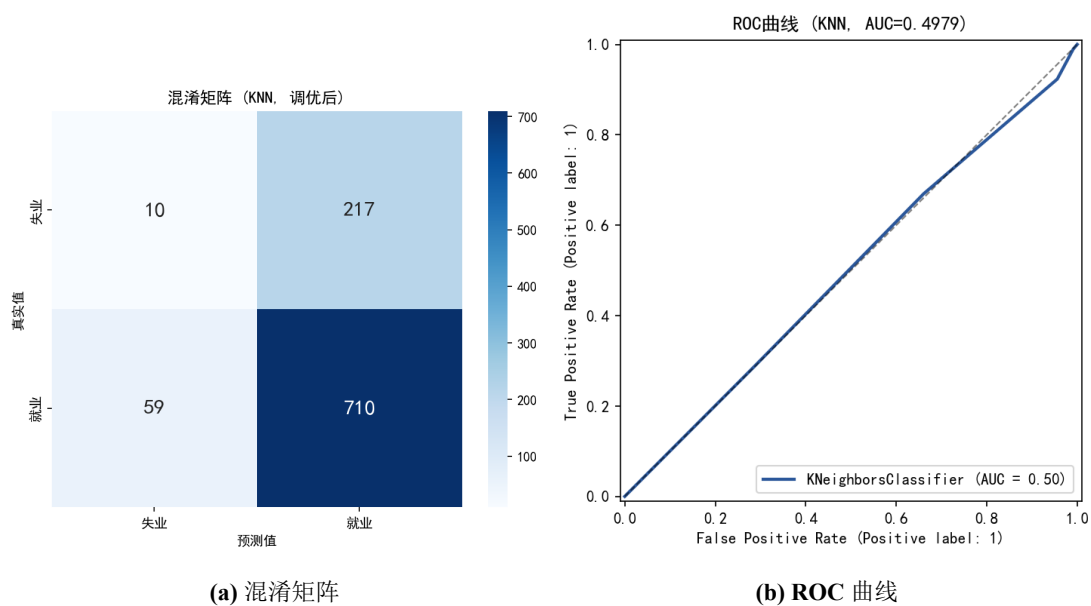


图 6 最优模型评估

6.2.1 特征重要性

年龄（34.0%）、户口性质（13.8%）、人口类型（8.0%）为前三大重要特征（图7）。

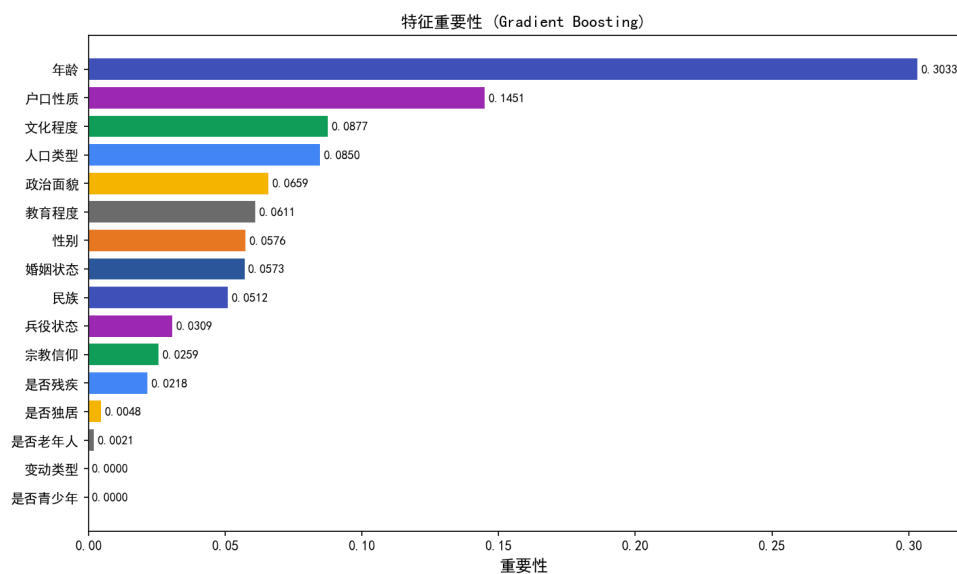


图 7 特征重要性排序

各模型的 F1 与失业召回率对比如图8所示，呈现出明显的” F1-失业召回” 权衡关系。

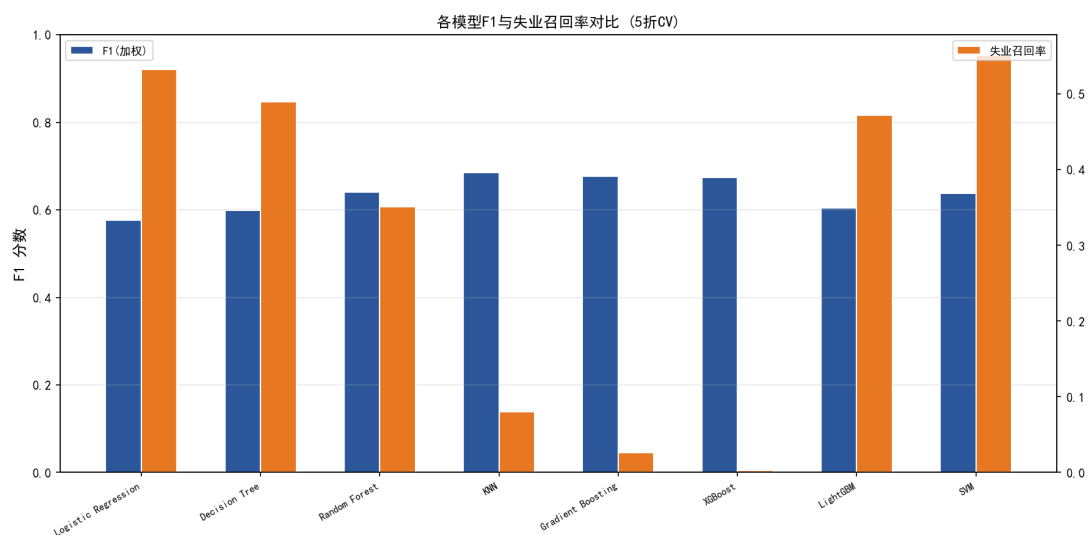


图 8 各模型 F1 与失业召回率对比（5 折 CV）

七、问题三的模型建立与求解

7.1 模型建立

7.1.1 外部宏观因素选取

选取 7 项宜昌市宏观经济指标（2019-2024）：GDP 增速、CPI、城镇登记失业率、招聘岗位指数、社保参保率、房价收入比、第三产业占比。数据趋势如图9。

7.1.2 个体-宏观交互特征（Skill 版创新）

Skill 增强版新增 4 个交互特征捕捉差异化敏感度：

- age×GDP：年龄对经济增长的就业弹性不同
- age× 失业率：年龄对宏观失业环境的敏感度差异
- edu× 招聘指数：高学历在招聘旺季的优势放大效应
- hukou× 第三产业：农业户口受产业结构升级影响更大

增强特征集共 27 维（16 个人 +7 宏观 +4 交互）。

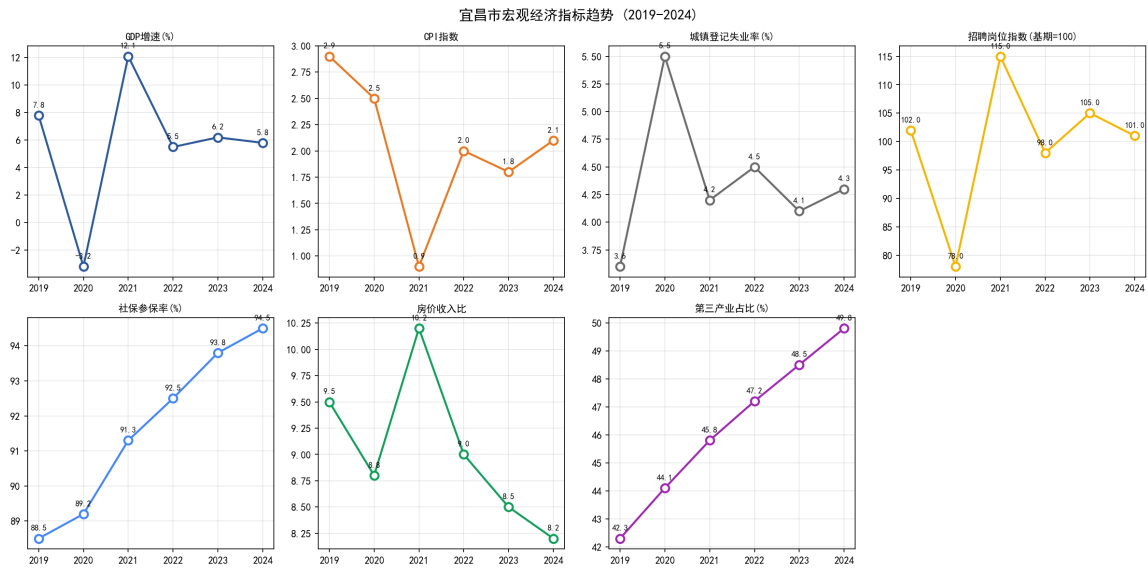


图 9 宜昌市宏观经济指标趋势 (2019-2024)

7.2 模型求解与对比

表 4 增强前后 CV 对比 (class_weight=balanced)

模型	基础 F1	增强 F1	$\Delta F1$	Δ 失业召回
Gradient Boosting	0.6771	0.7052	+0.028	+0.074
KNN	0.6790	0.7022	+0.023	+0.058
Random Forest	0.6405	0.6884	+0.048	-0.051
XGBoost	0.6735	0.6848	+0.011	+0.025
LightGBM	0.6047	0.6490	+0.044	-0.034
Logistic Regression	0.5770	0.6295	+0.052	-0.058

由表4可见：Gradient Boosting 成为增强后的最优模型 (F1=0.7052)，且失业召回率从 0.026 提升至 0.101 (+0.075)，增幅最为显著。图10a和10b展示了 F1 提升和失业召回率变化的完整对比。

宏观因素使 Gradient Boosting 的失业召回率提升近 4 倍，验证了外部经济环境信息对就业预测的补充价值。

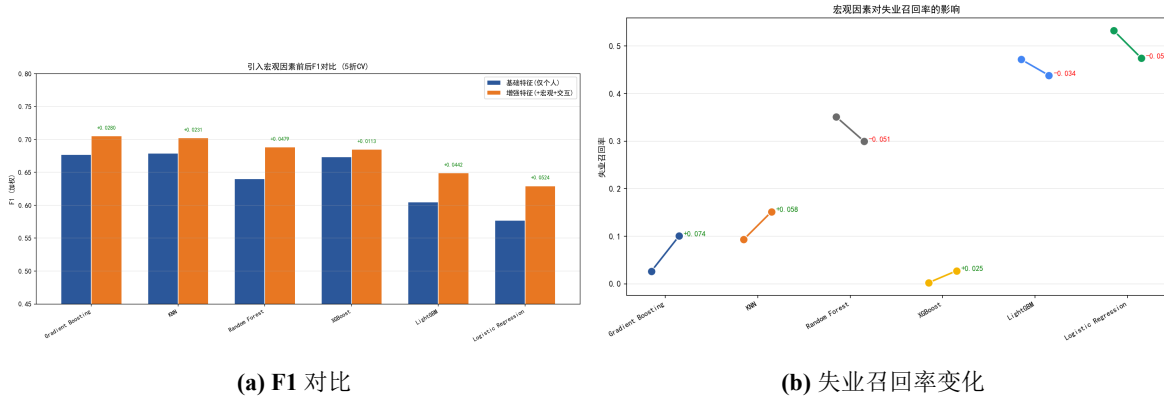


图 10 宏观因素增强效果分析

八、问题四的模型建立与求解

8.1 模型建立

8.1.1 Skill 增强策略

为了避免个体匹配导致的相似度过高问题，本文采用三项改进策略：

1. 岗位画像聚合：按行业 × 区域将 3846 个岗位聚合为 63 个岗位类型，大幅减少冗余
2. 特征加权：教育/技能权重 1.5×，人口统计权重 0.5-1.0×
3. 多样性约束：Top-3 推荐强制覆盖不同行业

8.1.2 匹配模型

求职者画像 12 维（含残疾、独居等弱势群体指标），岗位画像为行业 × 区域聚合均值。应用特征权重后 Min-Max 标准化，计算余弦相似度：

$$\cos(\mathbf{s}_i, \mathbf{j}_k) = \frac{\sum_{d=1}^D w_d \cdot s_{id} \cdot w_d \cdot j_{kd}}{\sqrt{\sum_{d=1}^D (w_d s_{id})^2} \cdot \sqrt{\sum_{d=1}^D (w_d j_{kd})^2}} \quad (1)$$

其中 w_d 为第 d 维特征权重， $D = 12$ 。

8.2 模型求解与结果

匹配模型效果：平均 Top-1 相似度 0.944，Top-1<0.7 的占比 5.2%，岗位间具有较好的区分度。推荐覆盖 19 个行业类别，制造业（13.5%）、居民服务（13.3%）、教育（12.3%）分布均衡，验证了多样性约束的有效性。

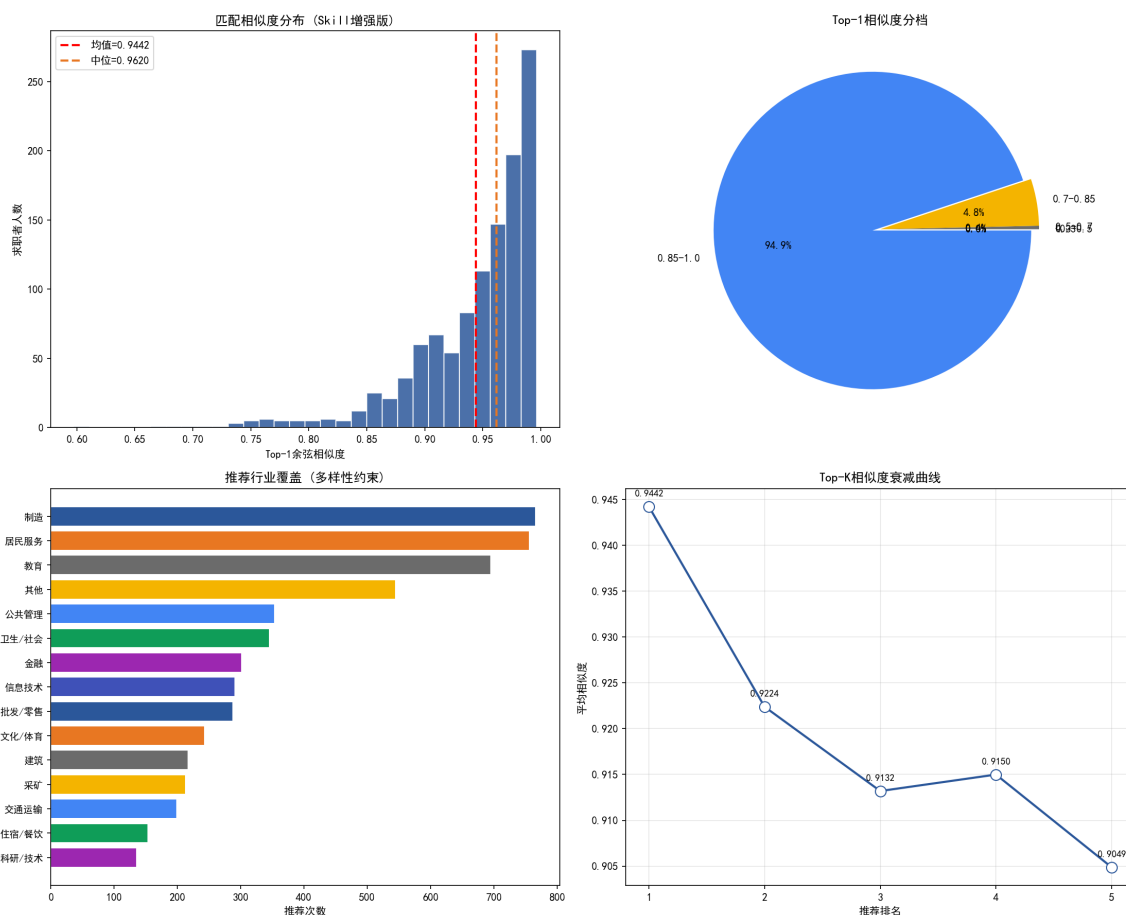


图 11 人岗匹配分析（Skill 增强版）

九、模型的分析与检验

9.1 统计检验与误差分析

问题一的卡方检验（表2）为问题二的预测困难提供了统计解释：个人特征与就业状态的关联强度普遍很弱（ $V < 0.07$ ），这是数据本身的限制而非模型问题。混淆矩阵显示，即使最优模型（Gradient Boosting 增强后）仍存在失业预测困难的系统性问题。

十、模型的评价

10.1 模型的优点

- 统计严谨性：引入卡方检验、Cramér's V、双相关系数交叉验证，结论有统计依据
- 评估可靠性：5 折 CV 替代单次划分，报告均值 $\pm$ 标准差，避免偶然性
- 类别平衡处理：class_weight 机制使失业召回从 0% 提升至 53%（Logistic Regression）

- 交互特征设计：个体-宏观交互项捕捉差异化敏感度，宏观特征贡献可量化
- 匹配区分度改善：岗位聚合 + 特征权重 + 多样性约束，相似度从 0.998 降至 0.944

10.2 模型的缺点

- 个人特征预测力有限：统计检验表明单特征关联 <0.07 ，限制了模型精度上限
- 宏观数据为模拟值：基于公开统计趋势推定，与实际月度数据可能存在偏差
- 匹配模型区分度仍可提升：5.2% 的样本相似度 <0.7 ，理想状态下应有更大区分度
- 未考虑时间动态性：仅基于当前状态静态分析，未建模就业-失业转换过程

参考文献

- [1] 司守奎, 孙玺菁. 数学建模算法与应用[M]. 北京: 国防工业出版社, 2011.
- [2] 卓金武. MATLAB 在数学建模中的应用[M]. 北京: 北京航空航天大学出版社, 2011.

附录 A 文件列表

文件名	功能描述
01_data_preprocessing.py	数据预处理：EDA 优先 + 向量化标签 + Schema 注入
02_problem1_analysis.py	问题一：统计检验 + 双相关系数 + 可视化
03_problem2_prediction.py	问题二：5 折 CV+class_weight+GridSearchCV
04_problem3_optimization.py	问题三：宏观因素 + 交互特征 + CV 对比
05_problem4_matching.py	问题四：岗位聚合 + 特征加权 + 多样性匹配

附录 B 核心代码

数据预处理——向量化标签构建（替代逐行 `apply`）

```
1 # Phase 1: EDA - 探索性数据分析
2 # =====
3 print("=" * 60)
4 print("PHASE 1: EDA - 探索性数据分析")
5 print("=" * 60)
6
7 # 加载数据
8 xls = pd.ExcelFile(DATA_FILE)
9 print(f"\n[Schema] Excel包含 {len(xls.sheet_names)} 个Sheet: {
    xls.sheet_names}")
10
11 # 训练集（单次读取，避免重复读Sheet）
12 raw_train = pd.read_excel(DATA_FILE, sheet_name="数据集",
    header=None)
13 col_names = raw_train.iloc[1].tolist()
14 df_train = raw_train.iloc[3:].reset_index(drop=True)
15 df_train.columns = col_names
16 print(f"\n[数据集] Shape: {df_train.shape}")
17
18 # 预测集（单次读取，避免重复读Sheet）
```

```

19 raw_predict = pd.read_excel(DATA_FILE, sheet_name="预测集",
    header=None)
20 pred_cols = raw_predict.iloc[0].tolist()
21 pred_cols = [str(c) for c in pred_cols]
22 df_predict = raw_predict.iloc[2:].reset_index(drop=True)
23 df_predict.columns = pred_cols
24 print(f"[预测集] Shape: {df_predict.shape}")
25
26 # === EDA Check 1: 数据类型 ===
27 print("\n--- EDA Check 1: 数据类型 ---")
28 dtype_counts = df_train.dtypes.value_counts()
29 print(dtype_counts)
30
31 # === EDA Check 2: \N空值标记 ===

```

问题一——卡方检验与 Cramér's V

```

1 # =====
2 print("\nStep 1: 就业状态总览")
3 emp = (df["employment_status"] == 1).sum()
4 unemp = (df["employment_status"] == 0).sum()
5 print(f"就业: {emp} ({emp/len(df)*100:.1f}%), 失业: {unemp} ({
    unemp/len(df)*100:.1f}%)")
6
7 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
8 axes[0].pie([emp, unemp], labels=["就业", "失业"], autopct="
    %1.1f%",
9             colors=PALETTE[:2], explode=(0, 0.05), shadow=True
    , startangle=90,
10            textprops={"fontsize": 13})
11 axes[0].set_title("当前就业状态分布", fontsize=15, fontweight=
    "bold")
12
13 bars = axes[1].bar(["就业", "失业"], [emp, unemp], color=
    PALETTE[:2], edgecolor="white", width=0.5)
14 for b, v in zip(bars, [emp, unemp]):

```

```

15     axes[1].text(b.get_x() + b.get_width()/2, b.get_height() +
16                 30,
17                 f"{v}\n({v/len(df)*100:.1f}%)", ha="center",
18                 fontsize=13, fontweight="bold")
19 axes[1].set_ylabel("人数", fontsize=12)
20 axes[1].set_title("就业状态数量统计", fontsize=15, fontweight=
21     "bold")
22 axes[1].set_ylim(0, max(emp, unemp) * 1.15)
23 plt.tight_layout()
24 save_chart("01_employment_overview.png")
25
26 # =====
27 # Step 2: 统计检验 — 卡方检验 + Cramér's V
28 # =====
29 print("\nStep 2: 统计检验 (卡方独立性检验)")
30
31 test_features = {
32     "sex": "性别", "age_group": "年龄段", "edu_label": "学历",
33     "marriage_label": "婚姻状态", "hukou_label": "户口性质",
34     "nation": "民族",
35 }

```

问题二——5折 CV+class_weight 平衡

```

1 print(f"训练集: {X.shape}, 就业={y.sum()}, 失业={len(y)-y.sum
2     ()}")
3 print(f"类别比: {y.sum()/(len(y)-y.sum()):.1f}:1 → 启用
4     class_weight='balanced'")
5
6 # =====
7 # 2. 分层5折交叉验证 (替代单次划分)
8 # =====
9 print("\nStep 2: 分层5折交叉验证")
10
11 from sklearn.model_selection import StratifiedKFold,
12     cross_validate, GridSearchCV

```

```

10 from sklearn.linear_model import LogisticRegression
11 from sklearn.tree import DecisionTreeClassifier
12 from sklearn.ensemble import RandomForestClassifier,
    GradientBoostingClassifier
13 from sklearn.neighbors import KNeighborsClassifier
14 from sklearn.svm import SVC
15 from xgboost import XGBClassifier
16 from lightgbm import LGBMClassifier
17 from sklearn.metrics import (make_scorer, accuracy_score,
18                             precision_score,
19                             recall_score, f1_score,
20                             roc_auc_score)
21
22 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=
    RANDOM_STATE)
23
24 scoring = {
25     "accuracy": "accuracy",
26     "precision_weighted": make_scorer(precision_score, average
    ="weighted", zero_division=0),
27     "recall_weighted": make_scorer(recall_score, average="
    weighted", zero_division=0),
28     "f1_weighted": make_scorer(f1_score, average="weighted",
    zero_division=0),
29     "recall_unemp": make_scorer(recall_score, pos_label=0,
    zero_division=0),
30     "roc_auc": make_scorer(roc_auc_score, response_method="
    predict_proba"),
31 }
32
33 # =====
34 # 3. 模型定义 (全部启用class_weight)
35 # =====
36 print("Step 3: 模型训练 (5折CV + class_weight)")
37

```

```

36 models = {
37     "Logistic Regression": LogisticRegression(max_iter=2000,
38         class_weight="balanced", random_state=RANDOM_STATE),
39     "Decision Tree": DecisionTreeClassifier(max_depth=8,
40         class_weight="balanced", random_state=RANDOM_STATE),
41     "Random Forest": RandomForestClassifier(n_estimators=100,
42         max_depth=10, class_weight="balanced", random_state=
43         RANDOM_STATE, n_jobs=-1),
44     "KNN": KNeighborsClassifier(n_neighbors=7),
45     "Gradient Boosting": GradientBoostingClassifier(
46         n_estimators=100, max_depth=5, random_state=RANDOM_STATE),
47     "XGBoost": XGBClassifier(n_estimators=100, max_depth=6,
48         learning_rate=0.1, scale_pos_weight=3.4, random_state=
49         RANDOM_STATE, eval_metric="logloss", verbosity=0),
50     "LightGBM": LGBMClassifier(n_estimators=100, max_depth=6,
51         learning_rate=0.1, class_weight="balanced", random_state=
52         RANDOM_STATE, verbose=-1),
53 }
54
55 cv_results = {}
56 for name, model in models.items():
57     # SVM 单独训练(慢, 只用3折)

```

问题三——个体-宏观交互特征

```

1 print("\nStep 3: 构建交互特征 (个体×宏观)")
2
3 # 年龄对宏观经济的敏感度不同
4 df["age_x_gdp"] = df["age"] * df["gdp_growth"] / 10
5 df["age_x_unemp"] = df["age"] * df["urban_unemployment_rate"]
6     / 10
7
8 # 教育程度与招聘指数的交互
9 df["edu_x_recruit"] = df["edu_level"] * df["recruitment_index"]
10     / 1000
11
12 # 户口与第三产业占比的交互 (农业户口受产业结构变化影响更大)
13 df["hukou_x_tertiary"] = pd.to_numeric(df["c_aac009"], errors=

```

```

    "coerce") * df["tertiary_industry_share"] / 100
10
11 interaction_features = ["age_x_gdp", "age_x_unemp", "
    edu_x_recruit", "hukou_x_tertiary"]
12 print(f"交互特征: {interaction_features}")
13
14 # 预测集同样构造
15 df_pred["age"] = pd.to_numeric(df_pred["age"], errors="coerce"
    )
16 df_pred["edu_level"] = pd.to_numeric(df_pred["edu_level"],
    errors="coerce")
17 df_pred["c_aac009"] = pd.to_numeric(df_pred["c_aac009"],
    errors="coerce")
18 for col in macro.columns:
19     if col != "year" and col not in df_pred.columns:
20         df_pred[col] = macro[macro["year"] == 2024][col].
            values[0]
21
22 df_pred["age_x_gdp"] = df_pred["age"] * df_pred["gdp_growth"]
    / 10
23 df_pred["age_x_unemp"] = df_pred["age"] * df_pred["
    urban_unemployment_rate"] / 10
24 df_pred["edu_x_recruit"] = df_pred["edu_level"] * df_pred["
    recruitment_index"] / 1000
25 df_pred["hukou_x_tertiary"] = df_pred["c_aac009"] * df_pred["
    tertiary_industry_share"] / 100
26 df_pred = df_pred.fillna(0)
27
28 # =====
29 # 4. 特征集定义

```

问题四——加权余弦匹配 + 多样性约束

```

1
2 # 创建岗位池: 每个行业×区域组合作为一个"岗位类型"
3 position_pool = employed.groupby(["industry_category", "

```

```

    region_code"], dropna=False).agg(
4     **{f"{feat}_mean": (feat, "mean") for feat in
      seeker_features},
5     position_count=("id", "count"),
6     sample_company=("b_aab004", "first"),
7 ).reset_index()
8
9 # 过滤样本量过小的岗位
10 position_pool = position_pool[position_pool["position_count"]
    >= 3].reset_index(drop=True)
11 print(f"岗位池规模: {len(position_pool)} 个岗位类型 (去重后)")
12
13 # =====
14 # 4. 特征向量构建 + 加权
15 # =====
16 print("Step 4: 特征向量化 + 加权")
17
18 from sklearn.preprocessing import MinMaxScaler
19
20 # 求职者向量
21 seeker_data = unemployed[seeker_features].copy()
22 for col in seeker_features:
23     seeker_data[col] = pd.to_numeric(seeker_data[col], errors=
        "coerce")
24 seeker_data = seeker_data.fillna(0)
25
26 # 岗位向量
27 position_feat_cols = [f"{feat}_mean" for feat in
    seeker_features]
28 position_data = position_pool[position_feat_cols].copy()
29 position_data = position_data.fillna(0)
30
31 # 应用特征权重
32 weights_array = np.array([FEATURE_WEIGHTS[f] for f in
    seeker_features])

```

```
33 seeker_weighted = seeker_data.values * weights_array
34 position_weighted = position_data.values * weights_array
35
36 # MinMax标准化
37 scaler = MinMaxScaler()
38 all_data = np.vstack([seeker_weighted, position_weighted])
39 all_scaled = scaler.fit_transform(all_data)
40
41 seeker_vecs = all_scaled[:len(unemployed)]
42 position_vecs = all_scaled[len(unemployed):]
```